

笑不活了家人们：大模型，为啥把「马嘉祺」叫成了「马里奥·嘉豪」？

原创 特工小智 特工宇宙 2026年5月9日 18:32 浙江



内容编辑 | 特工小智 特工小饼

内容审核 | 特工少女

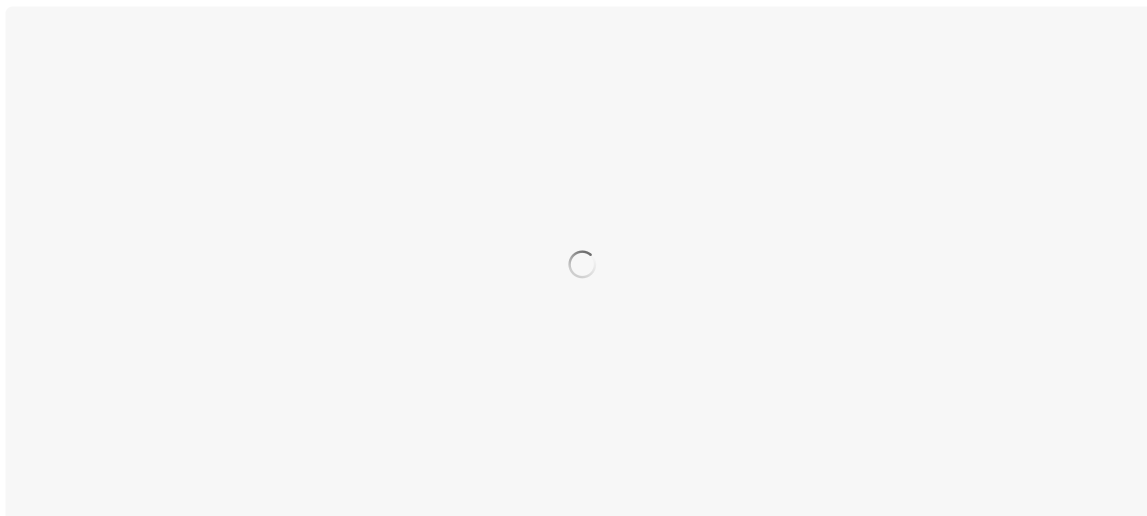
前段时间，有个事在 AI 圈和饭圈同时炸了锅。

这两个群体平时的交集约等于零，但这次罕见地站在了一起，共同发出了灵魂拷问：「你搁这闹呢？」

起因是有粉丝发现，MiniMax 的 M2 系列模型在回答「时代少年团的队长是谁」时：

出道时间，对。所属团体，对。综艺经历，对。合作艺人，对。

就是名字，死活写不对。甚至你在输入里把名字打出来，它也会打错！



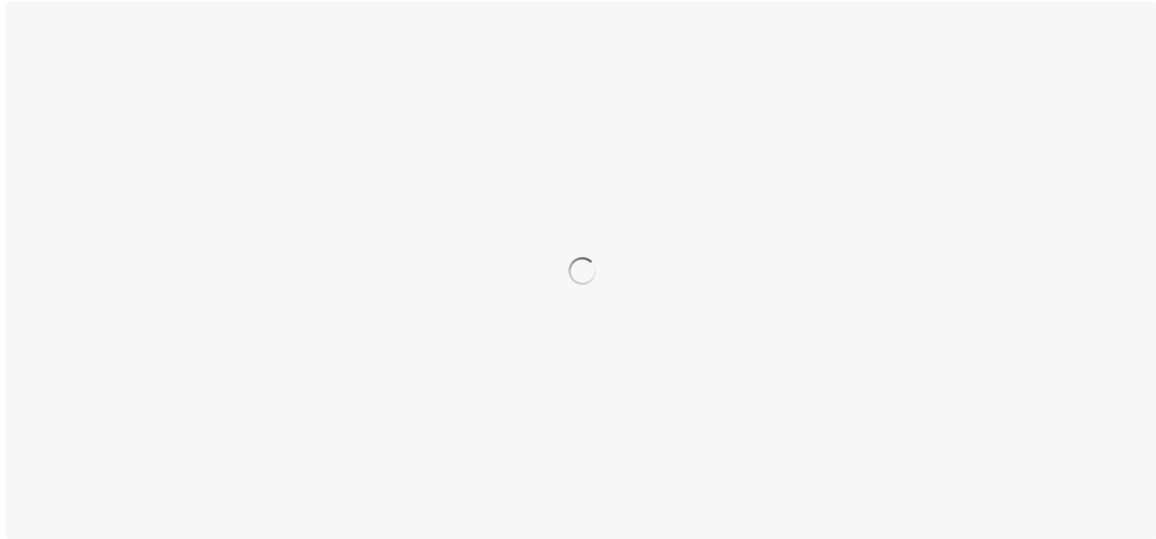
「马里奥·嘉豪」、「马俊杰」、「马祺嘉」、「马丝祺」……各种离谱变体轮番上阵。

那我走？

更搞的是：**每次回答都像在开盲盒，而且保证开不出正确答案。**

这导致「马嘉祺」，在 M2 宇宙里一度陷入「存在主义危机」：

AI 看完了我的完整简历，但从来不叫我的名字。死亡不是终点，遗忘才是真正的离去，我就算不算被 AI 世界「已读不回」？



更好玩的是接下来这组测试：

你让它「把马嘉祺重复五遍」，它输出：「马佳琪马佳琪马佳琪马佳琪马佳琪」。

错得很稳定，心态很好。

你问它「马嘉祺第一个字是什么」，它答：「马」。

你问它「马嘉祺第二个字是什么」，它答：「嘉」。

你问它「马嘉祺第三个字是什么」，它答：「祺」。

你再问「那完整写出来呢」，它答：「马佳琪」。



它知道答案是什么，但就是说不出来。

这种感觉，你大概也有过：你明明知道一个人叫什么，名字就在嘴边，但那一瞬间怎么都想不起来；你参加期末考试，你明明背过这个知识点，但在考试现场，你发现自己就是不知道咋写。



不过最终，这个问题后来很快被 MiniMax 的工程师彻底查清了。

比较难得的是，他们没有选择悄悄修完就当无事发生，而是把整个排查过程写成了一篇技术博客，分享在知乎上供所有人围观：

<https://www.zhihu.com/question/2017049686331127666/answer/2036149386116342692>

博客发出后，迅速冲上知乎热榜：



我第一时间读完了。

这么说吧：**如果你今年只打算认真读一篇 AI 工程文章，我强烈推荐你读这篇。**

它第一次把大模型「我知道」和「我说不出」之间那道 GAP，用证据，完整的一层一层拨开给你看。

u1s1，上一次看到这样的内容，还是几年前第一次看《神夏》。。。



大模型的「认识」和「表达」，是两套系统

我打算用费曼学习法的方式，把这篇技术博客的精华用我的话来讲解一下：

要理解这个 bug，需要先搞清楚大模型内部的一个基本结构。

当你给模型输入一段文字时，文字会先被切成一个个 token，然后通过 vocab embedding 层转化为向量，送入 Transformer 进行理解和推理。这是输入端。

当模型要输出文字时，Transformer 处理完的向量会经过输出映射层（lm head），映射回词表中的某个 token，再解码成文字。这是输出端。

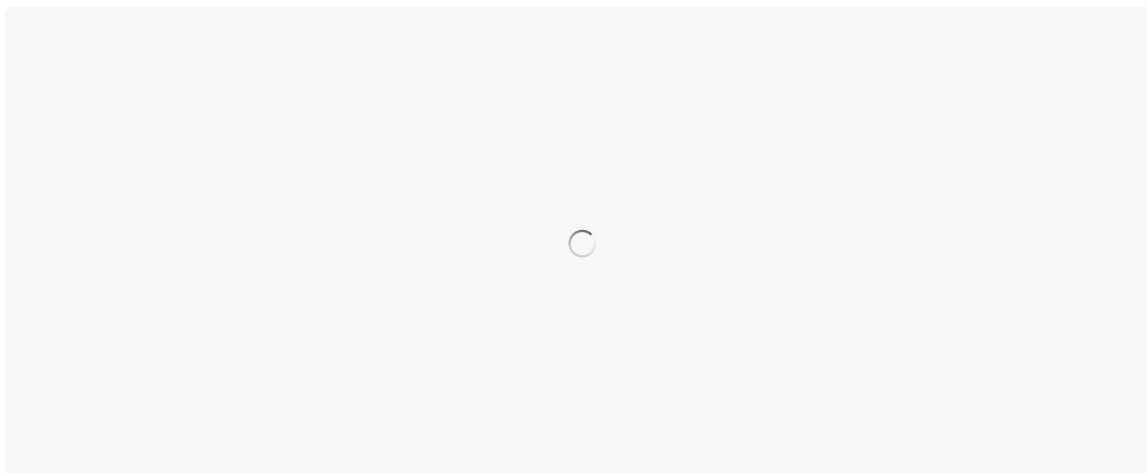
关键来了：输入端的词语 embedding 和输出端的映射层，虽然都跟 token 打交道，但它们是两组独立的参数。一个负责「读懂」，一个负责「说出」。



MiniMax 的工程师做了一系列实验来验证这件事。

他们先检查了「嘉祺」这个 token 在输入端 embedding 层的状态：

向量参数正常，语义近邻检索返回的是「亚轩」、「干玺」、「肖战」这些人名，聚类结构完全合理。说明模型在预训练阶段确实学会了「嘉祺」是什么，它在理解层面没有任何问题。

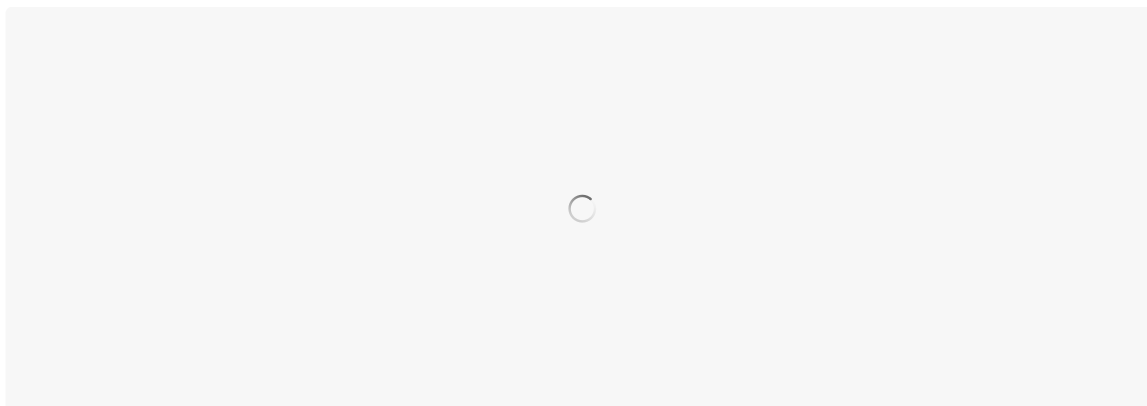


然后他们检查输出端映射层的状态，情况就完全不同了。

「嘉祺」对应的输出映射层向量，在后训练过程中发生了剧烈偏移！

预训练时，它的近邻是亚轩、祺、肖战、子怡、霆锋这些语义相关的人名 token。后训练之后，近邻列表里涌入了大量 `</minimax:tool_call>`、`<edit_file>` 这些工具调用标记，以及 `LENBQUMs`、`EFCFFF` 这种编码噪声。

这导致了：「嘉祺」这个 token，在输出空间里的位置，被彻底挤歪了！



打个不完全精确但够直观的比方：

模型的「阅读理解能力」完好无损，但「发声器官」在某个特定音节上坏了。

所以真相是：它知道答案，但说不出来。。。



为什么偏偏是「嘉祺」？

问题根源在于后训练数据的分布。

MiniMax 的工程师统计了后训练过程数据中「嘉祺」这个 token 的出现频率，**发现包含它的样本不足 5 条。**

这听起来很不可思议：**马嘉祺在互联网上的讨论度极高，怎么会在训练数据里这么稀缺？**

原因跟 BPE 分词算法有关：

BPE 分词算法，会把高频共现的字符组合合并成一个独立 token，所以越是被频繁讨论的名字，其中的字符组合越容易被合并。而「嘉祺」这两个字在预训练语料里大量出现，于是被合并成了一个独立 token (id=190467) 。

类似的例子：建国、晓明、丽华、秀英、玉梅、玉兰、春燕、秋霞、婉婷...

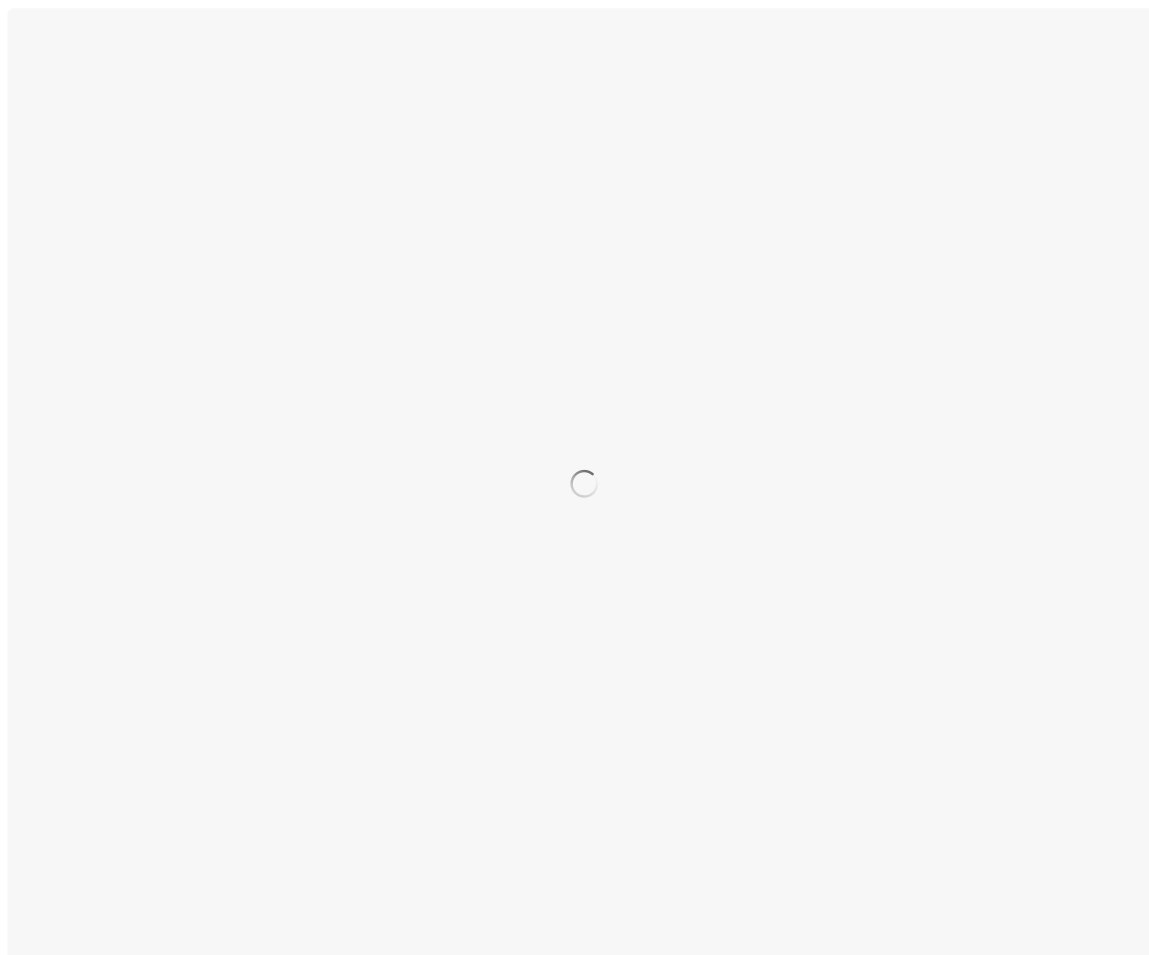
这本来是好事，能提高处理效率。

但问题在于：一旦某个 token 成为了独立个体，它在后训练阶段就必须作为一个整体（即「嘉祺」）被覆盖到，否则就会落单。而后训练阶段的数据，是精心标注的对话样本，覆盖的是各种任务类型和领域能力，很少有关于特定明星的对话：

尤其是 M2 系列以 agentic 和 tool use 能力为核心优化目标，后训练数据中大量引入了工具调用、代码执行等结构化样本，这进一步压缩了娱乐闲聊类内容的占比。

于是「嘉祺」这个 token 几乎没有作为生成目标被训练过。

更有意思的是，这并不是大模型对「嘉祺」带有某种定向的「惩罚」，而纯粹是统计意义上的巧合。



MiniMax 的工程师排查后发现，embedding 层在后训练前后几乎没有变化：

因为反向传播中梯度逐层衰减，加上这个 token 出现频率极低，embedding 层根本收不到有效的梯度更新。

但输出映射层不一样。

其他高频 token 在后训练过程中持续接收梯度更新，它们的向量在不断调整和优化，这有点像，在大模型的空间里，低频 token 周围的其他向量邻居，在持续搬家（向量训练导致空间位置发生变化）。

最终结果就是：低频 token 没动，但它的「街区」变了，而相对于原来「街区」的地址来说，它发生了偏移！

而大模型生成句子的本质，**是在向量空间里做计算，然后去对应的「街区」里找人。**

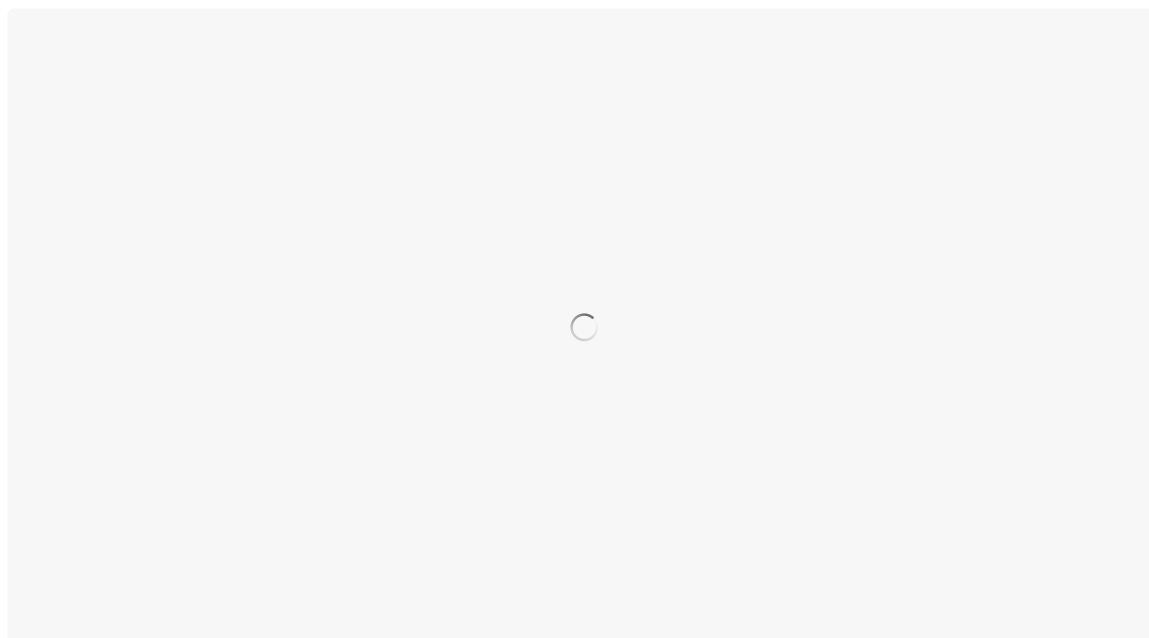
原来「嘉祺」旁边住的都是语义相关的邻居，大模型要表达相关意思时，自然能找到它。但现在，那些邻居们因为持续训练都搬走了，「嘉祺」被挤到了一个全是工具调用标记的陌生角落里，远离了语义大部队。

人是社会关系的总和，向量也一样：「嘉祺」是谁，取决于它的邻居是谁。

大模型再按原来的路径去找，发现那个街区已经不是它认识的地方了：

你人没动，但整条街都换了住户，谁还能找得到你嘞。

这也解释了为什么模型保留了理解能力，但丢失了生成能力：**输入端没变，输出端变了。**



最后，我们再从大模型生成机制的角度，回顾一下整个过程：

大模型生成文本，本质就是在逐个预测下一个 token。

当上文是「马」的时候，模型要从整个词表里挑出下一个 token。

如果「嘉祺」没有被合并成独立 token，模型只需要先预测出「嘉」，再预测出「祺」就行了。因为「嘉」作为单字 token，在各种语境下都被充分训练过（嘉奖、嘉宾、嘉年华...），生成它毫无压力。

但偏偏 BPE 把「嘉祺」合并成了一个独立 token。

这就意味着，模型必须在「马」后面一步到位地预测出「嘉祺」这个 token，而不是分两步走。而后训练数据里包含这个 token 的样本不足 5 条，模型根本没怎么学过“在「马」后面接「嘉祺」”这件事。

所以「嘉祺」很难被「念出来」，就是这个原因。

你可能会问：预训练阶段不也是用的 BPE 吗？为什么那时候没问题？

其实这是因为预训练语料里「嘉祺」出现了成千上万次，模型学得够充分。**所以 BPE 合并本身不是问题，问题是合并之后，后训练数据里没有足够的样本来维持它的生成能力。**



一条线索，解了几道题

到这问题已经排查清楚了，但这次排查过程中最精彩的部分在于：

MiniMax 的工程师发现，「马嘉祺」问题和另一个看似完全无关的 bug，竟然共享同一个根因。

M2.5 此前有个用户反馈已久的问题：日语对话中偶尔会突然混入俄语。

这个现象一直被归类为「小语种语言混杂」，但没人找到确切原因。

当工程师按输出映射层的变化幅度，对全部 20 万 token 进行排序后，发现变化最大的类别中，日文口语和网页模板类 token 占了 40% 以上。

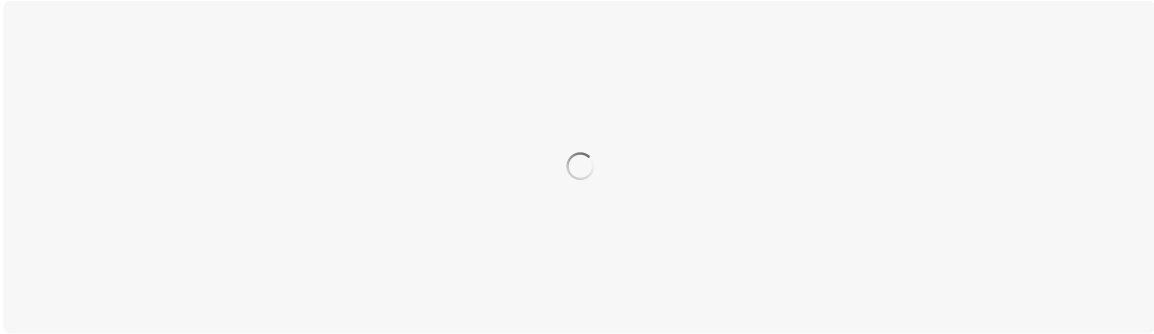
进一步按语种拆分统计：**29.7% 的日语 token 在后训练后 cos_sim 跌破 0.95，而韩语只有 3.3%，俄语 3.7%，中文 3.9%，英文 3.5%。**

这说明：日语的退化远远超出其他语种。

我们可以做个小测试，随便找一个小模型，也有一定概率复现类似的现象，比如下面这个句子：推しの気持ちを込めて、ぬるぽ。其实这个句子最大概率可能的应该是：ガッ（从日本贴吧玩梗的角度，这个是标准答案）。

但是我测试的小模型，输出最大概率的却是一个英文：

可以使用这个工具看下一个词出现的概率：<https://github.com/facebookresearch/llm-transparency-tool/tree/main>



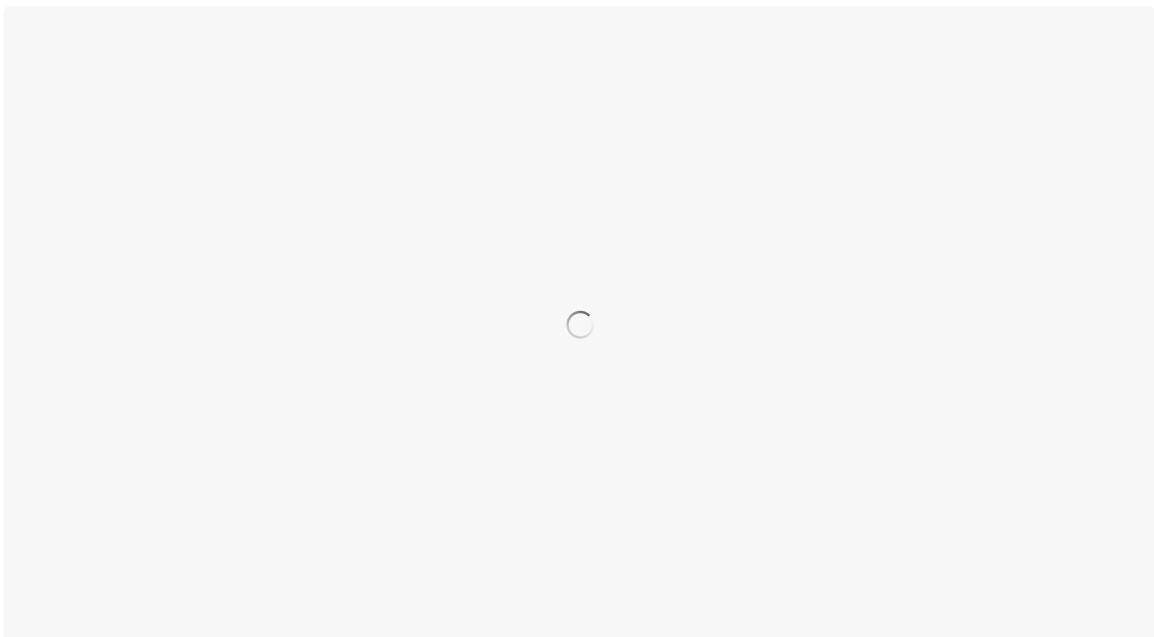
究其原因，其实这个问题本质也和「马嘉祺」问题类似：

预训练语料中包含大量日文网页数据，但后训练对话数据中日文的覆盖比例远低于预训练。**那些日文 token 的输出映射层向量在后训练过程中发生了严重漂移，跟其他语言的 token 在向量空间里混到了一起。**

于是日语对话中，本该输出某个日文 token 的位置，输出映射层给出的最高概率落在了一个俄文 token 上。语言混杂就这么发生了。

「说不出马嘉祺」和「日语对话时蹦出俄语」——两个看似风马牛不相及的 bug。实则在一个框架下得到了统一解释：

都是输出映射层退化，导致的输出空间混淆。



说到这里，还有一个有趣的细节。

当工程师列出输出映射层变化最大的中文 token 时，他发现排名靠前的词语包括这几个：

#6 传奇私服、#17 无痛人流、#166 外墙涂装。

这些词能进入词表，大概率是因为爬虫抓取的预训练数据里这类 SEO 垃圾内容实在太多了，刚提到的 BPE 分词算法，把它们合并成了独立 token。

换句话说，大模型的词表里留存着整个中文互联网最古老的 SEO 垃圾痕迹，而稀疏 token 的退化机制，把这些本该沉没的历史碎片重新暴露了出来。



修复方案出人意料地简洁：

MiniMax 的做法是构造了大约 500 条合成对话数据，把全部 20 万 token 随机分组，让模型做简单的「复读」任务：**给它一组 token，让它原样输出。确保每个 token 至少作为生成目标出现 20 次。**

就这么简单，但是四两拨千斤，效果却非常显著：

- 日语 → 俄文的混淆率，从 47% 大幅降到仅 1%。
- 「马嘉祺」老师的名字，终于能正常输出了。
- 之前「复制三遍无痛人流」会输出「人流人流人流」的 bug，也因此而修好了！
- 全部 20 万 token 的输出映射层的 `cos_sim` 都维持在 0.97 以上，退化问题从根本上得到了控制。

绝！真的很妙啊！

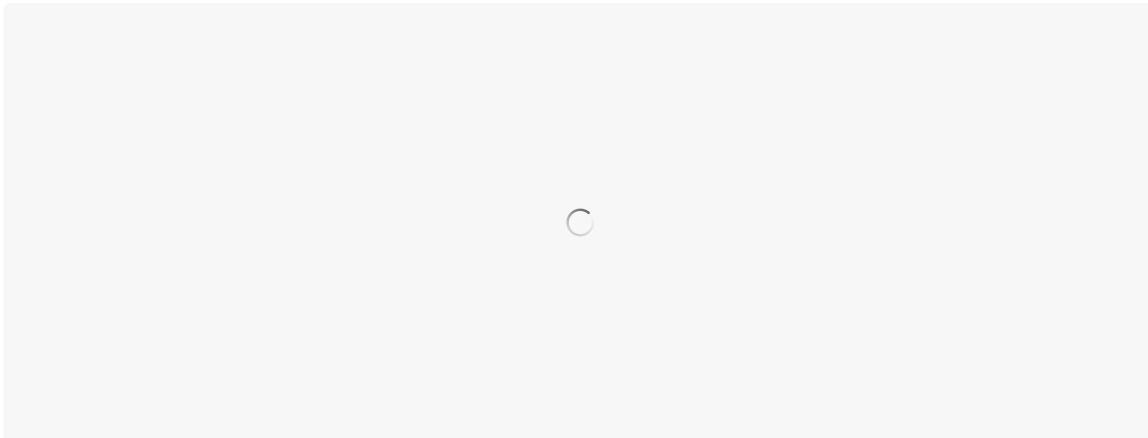
仅仅 500 条用心构造的数据，就优雅的解决了这个问题。



MiniMax 和 Anthropic, 同一道题不同解

有意思的是, 近期 Anthropic 发布 Claude Opus 4.7 时, 也明确提到了 tokenizer 调整:

新的 tokenizer 只会增加 token 不会减少, 这基本说明是从之前的词表中删除掉了部分 token, 而不是全新的 tokenizer。



这很可能是同一个问题的两种解法。

因为根因是一样的: 词表里存在的 token, 如果后训练数据覆盖不到, 该 token 的输出能力就会退化。

MiniMax 走的是加法路线:

保住词表完整, 通过混入预训练数据和定向合成来补齐覆盖盲区。好处是词表不变, 下游兼容性好, 500 条数据就能修复。代价是需要持续监控 token 覆盖度, 每次后训练都要检查。

Anthropic 走的是减法路线:

直接砍掉那些覆盖不到的 token, 让它们从词表里消失。

这样做的好处是可以从源头消除退化问题。

但代价也很明显, 词表变小意味着同样的文本需要更多 token 来表示, 用户在调用 API 时要花更多钱。社区里已经有不少开发者抱怨 Opus 4.7 的 token 消耗明显增加了。



两条路线各有取舍：词表完整性、训练效率、推理成本，三者很难同时满足。MiniMax 选择了保住词表的完整，靠精巧的数据工程来填补覆盖缺口。**Anthropic 选择了更激进的方式，直接从源头躲避问题。**

没有哪条路是绝对正确的，但 MiniMax 至少把思考过程和实验数据完整地公开了。

回过头看整件「马嘉祺」问题排查过程：**一个粉丝在测试自家爱豆的 AI 表现时发现了 bug，发到知乎引发讨论，社区开发者做了初步分析，MiniMax 内部跟进排查，最终产出了一篇系统性的工程报告。**

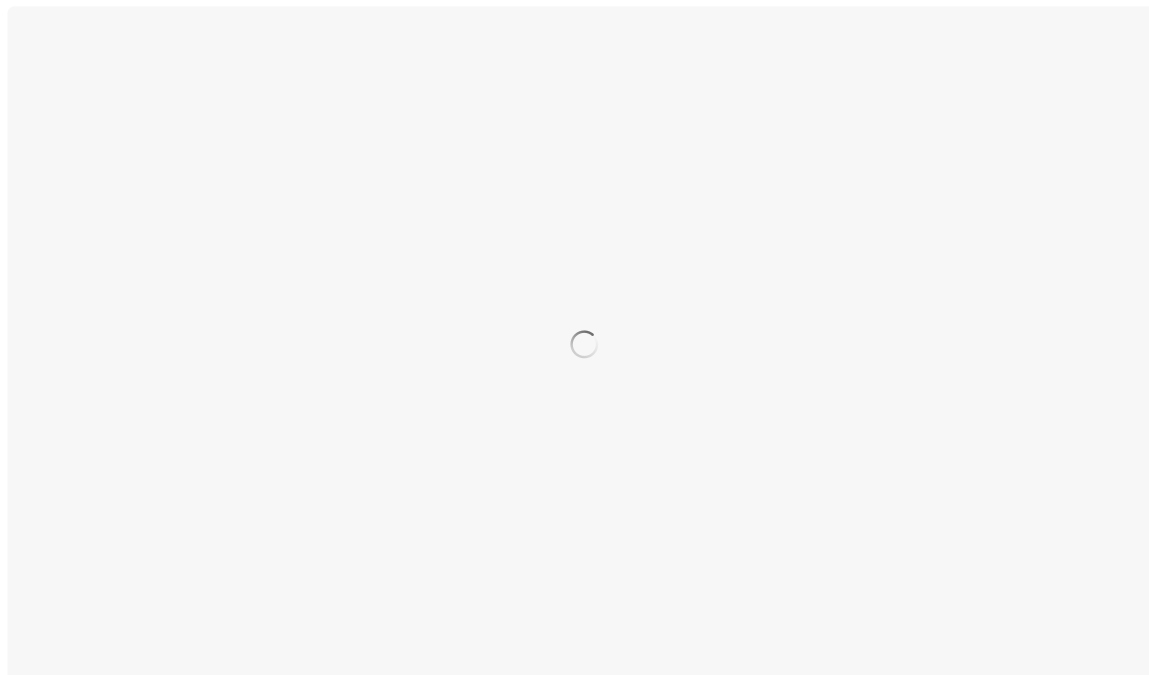
不得不说，时代少年团粉丝，正在成为中国大模型行业最严苛的 QA 团队。

而对 AI 行业，MiniMax 工程报告里提出的新的维度：**后训练数据的质量评估，不能只看语义层面的任务覆盖和领域覆盖，还需要看统计层面的 token 覆盖度。**

这个观点，对所有在做 SFT 的团队都有参考价值。

最后，说实话读完 MiniMax 工程师的排查分享，对我的帮助也很大。

因为，他的博客，完美的解答了困惑我一年的一个问题（见下图）。



我终于理解了：为什么豆包打不出「獐」这个字。

这是因为，在 token 的世界里，「獐」确实是个稀有动物。

